Reducing Inference Latency in Lightweight, Privacy-Preserving Email Triage: A Case Study on Skyler AI

Author: Sanjay Kumar

Project: Skyler AI (Skyler Inbox) — Gmail triage, classification and AI-assisted reply tool

Category: Applied systems / Natural Language Processing (NLP) for productivity software

> **Note to self before submission:** this draft is written to be adapted for a specific conference's template (IEEE/ACM two-column, Springer CCIS, etc.). Re-check author affiliation, page limits, and citation style (this draft uses an informal author-year style) against the target venue's call for papers before submitting.

Abstract

On-device and small-model Natural Language Processing (NLP) pipelines are increasingly used for privacy-preserving email triage, summarization, and classification, as an alternative to routing message content through third-party cloud LLMs. We take as a direct, verifiable baseline three freely-accessible papers that each evaluate one part of this same problem space — content-based email triage-action prediction (Mukherjee & Jiang, 2019, arXiv), abstractive email-thread summarization (Zhang et al., 2021, ACL — EmailSum), and transformer-based business-email workflow classification (Prexawanprasut, 2025, Thailand Statistician) — and show that all three, like the broader literature they represent, validate their own single task *offline*, on a fixed dataset, never the *request-time serving lifecycle* of a model once it is embedded in a live backend that has to combine several signals into one product decision. A separate line of systems research studies cold-start latency for large-scale LLM serving infrastructure (GPU clusters, edge-distributed pipelines), but again not for small in-process models owned by ordinary application code. We identify and fix exactly this gap in Skyler AI, a production Gmail inbox-organization application that filters, prioritizes, and semantically matches up to 200 emails per user action using local DistilBERT/MiniLM models: a warm, process-wide model singleton replacing per-request `pipeline()` re-initialization, in-app latency telemetry (`classifyMs`) so the effect is observable on real traffic rather than only in a benchmark script, and a reproducible, checked-in benchmark harness that drives the *actual* production classifier at realistic scale (200 emails/request, matching the product's own stated limit). In an isolated, mechanism-level measurement, eliminating per-request re-initialization removes 252–310 ms of pure model-loading overhead that was previously paid on *every* classification request — this is the paper's one clean, repeatable, high-confidence result. We also measured, and report honestly, two results that did **not** survive contact with realistic data: a batched-embedding optimization that gave a clean 1.55x speedup on *uniform-length* synthetic text collapsed to a wash (0.97x) on realistic, *variable-length* email sentences because of padding overhead, and a batched-sentiment-analysis optimization was found to be a consistent ~8% **regression** on this backend and was reverted from the shipped code rather than kept for an inflated headline number (a follow-up, length-bucketed batching strategy, described in the Addendum, later recovers a real benefit from both). At the full, realistic 200-email end-to-end scale, three independent benchmark runs on this (shared, sandboxed) machine gave noisy, inconsistent mean speedups (1.37x, 1.10x, 1.01x) — but a consistent, much larger reduction in tail-latency variance (1.2x–8.2x lower standard deviation), meaning the fix's clearest real-world benefit at this scale is eliminating severe latency spikes, not a fixed multiplier on the average case. We also measured Skyler AI's own deployed sentiment model directly against the real, official SST-2 validation benchmark (872 labeled examples) rather than leave a classification-quality comparison blank: 89.91% accuracy, 90.27% F1 — a real number on a different task than any of the three baseline papers, reported as such rather than implied to be equivalent. We discuss why this gap exists in the literature, what we found when we tried to validate our own optimizations rigorously, and what a fully rigorous follow-up evaluation would still need.

Keywords: email classification, semantic search, on-device NLP, transformer inference latency, cold-start, model lifecycle management, privacy-preserving AI

---

1. Introduction

Email remains the primary communication channel for most knowledge work, and inbox overload is a well-documented productivity problem. A growing category of tools — Skyler AI among them — try to address this by classifying, filtering, and drafting replies to email using a mix of lightweight local NLP models and cloud large language models (LLMs).

Skyler AI's current design (described in Section 4) intentionally avoids sending full email content to a third-party LLM for *every* message: it first runs a local sentiment classifier (`Xenova/distilbert-base-uncased-finetuned-sst-2-english`) and a local sentence embedder (`Xenova/all-MiniLM-L6-v2`), both running in-process via `@xenova/transformers` (a WASM/ONNX runtime for the browser and Node.js), and only calls a cloud LLM (Groq's `llama-3.1-8b-instant`) for the smaller task of summarizing and drafting a reply to emails the user has already chosen to act on. This is a privacy- and cost-motivated design: the local models never leave the server process, and they are cheap enough to run per-request without a GPU.

During real usage, however, the application became noticeably slow under its own "pull up to 200 mails" feature when an AI filter rule was attached to an inbox. This paper documents the investigation of that slowdown, connects it to a real gap in the published literature on small-model NLP inference, and presents and evaluates a fix implemented directly in the Skyler AI codebase.

Contributions:

1. We compare Skyler AI's local classification pipeline directly against three verifiable, freely-accessible papers covering triage-action prediction, thread summarization, and business-workflow classification — the same problem space split across three single-task evaluations — and show precisely what dimension all three leave unaddressed: request-time serving lifecycle and combined, multi-signal decision accuracy, as opposed to offline, single-task accuracy.

2. We identify a specific, reproducible inference-lifecycle bug — repeated cold-start of local transformer pipelines on every API request — in a real, deployed application, and show it is distinct from the cold-start problems studied in current LLM-serving literature.

3. We implement and ship a fix for that bug: a process-wide warm model singleton, inside `apps/api/src/AI_Services/gmail_AI/emailClassifier.ts` and `apps/api/src/AI_Services/models.ts`.

4. We add a small, shipped in-app telemetry feature (`classifyMs`, Section 7.3) so the fix's effect is visible on real traffic, not only in an offline benchmark — directly answering the "no end-to-end production measurement" limitation of a benchmark-only evaluation.

5. We build a reproducible, checked-in benchmark harness (`apps/api/src/scripts/benchmarkClassifier.ts`) that drives the real production classifier at realistic scale, and a unit test suite that pins down filtering correctness through the refactor.

6. We report, as a result in its own right, that a second optimization we tried (batching sentence/keyword embedding calls) only helps on unrealistically uniform-length text and a third (batching sentiment-analysis calls) was a measured regression and was reverted — an honest account of what *didn't* work, not just what did.

---

2. Related Work and Research Gap

We ground this paper in three verified, freely-accessible papers as a direct baseline (Section 2.1), then widen out to the broader small/local-model literature (Section 2.2) and the LLM cold-start/serving literature (Section 2.3). All bibliographic details below (title, authors, venue, metrics) were confirmed against the publisher's own page (arXiv, the ACL anthology, or the journal's own open-access platform) rather than taken from a search snippet, and all three are downloadable by anyone, free, with no institutional login.

2.1 Direct literature baseline

Mukherjee & Jiang (2019), *"A Content-Based Approach to Email Triage Action Prediction: Exploration and Evaluation"*, arXiv:1905.01991 (cs.IR). This paper frames the core decision every inbox-management tool like Skyler AI has to make — will the user act on this email? — as a learned ranking problem: it represents a user by the content of their current and past emails, adds similarity features between a user and an incoming email, and evaluates on the public Avocado email collection. Its best configuration (an ensemble of a TF-IDF/GBDT model and a CNN-MLP model, both using the similarity features) reaches **0.788 AUROC**, against a 0.757 best-baseline deep recommendation method.

Zhang, Celikyilmaz, Gao & Bansal (2021), *"EmailSum: Abstractive Email Thread Summarization"*, ACL-IJCNLP 2021 / arXiv:2107.14691. This paper introduces a human-annotated dataset of 2,549 email threads with short and long reference summaries, and benchmarks extractive and abstractive models (including fine-tuned T5) against it. The authors explicitly decline to report a single trusted automatic accuracy number: they show common metrics (ROUGE, BERTScore) correlate only weakly with human judgment on this task, and argue the field needs better evaluation methods — itself an honestly-reported finding, directly relevant to Skyler AI's own AI-summary/reply-draft feature.

Prexawanprasut (2025), *"Transformer-Based Email Classification for Workflow Automation in Small and Medium-Sized Enterprises"*, Thailand Statistician (open access, no paywall, via the TCI-Thaijo national journal platform). This paper classifies 12,500 real small-business emails (sent January 2021–December 2024) into four operational workflow categories, comparing a keyword-rule system, classical ML classifiers, and transformer models (BERT, DistilBERT, XLM-R). The best model, XLM-R, exceeds **92% accuracy**, and the authors integrate it into a prototype real-time workflow system that also extracts structured fields (invoice numbers, shipment codes, client IDs). Of every paper we found, this is the closest task-level match to what Skyler AI actually does: classify real business email for an inbox-management workflow.

What these three papers measure, and what they don't:

	Mukherjee & Jiang (2019)	**Zhang et al. (2021, EmailSum)**	**Prexawanprasut (2025)**	**This paper (Skyler AI)**
Model family	TF-IDF/GBDT + CNN-MLP ensemble, with similarity features	T5/BART abstractive summarization	BERT, DistilBERT, XLM-R	DistilBERT + MiniLM
Task	Predict user triage action (e.g., reply)	Email thread summarization	Business email classification (4 workflow categories)	Sentiment + semantic/keyword filtering + reply drafting
What's reported	0.788 AUROC (best), vs. 0.757 best baseline, on the Avocado dataset	No single trusted automatic metric; authors show ROUGE/BERTScore correlate weakly with human judgment	XLM-R > 92% accuracy, 12,500 emails, 4 domains	Per-request *inference* latency, before/after fix
Classification quality, measured by us	— (different task, own dataset)	— (different task; no comparable number exists even in the source paper)	— (different task: workflow category, not keep/discard)	**89.91% accuracy, 90.27% F1** — measured directly (Section 7.4), on the real, standard SST-2 validation set (872 examples), using the

	Mukherjee & Jiang (2019)	Zhang et al. (2021, EmailSum)	Prexawanprasut (2025)	This paper (Skyler AI)
				exact deployed sentiment model, not a self-reported number
Claims "production/real-time" suitability?	No explicit claim; offline evaluation only	No explicit claim; offline evaluation only	Yes — integrated into a prototype real-time workflow system	Measured directly, from a shipped backend
Measures per-request model lifecycle (cold start vs. warm reuse)?	**No**	**No**	**No**	**Yes — this is the paper's contribution**
Measures batched vs. sequential inference calls?	**No**	**No**	**No**	**Yes**
Measures a combined, multi-signal decision (not one model's label)?	**No**	**No**	**No**	**Yes**
Freely downloadable, no institutional login?	**Yes** (arXiv)	**Yes** (arXiv / ACL anthology)	**Yes** (open-access journal)	n/a

The gap is now concrete rather than inferred: all three papers validate one offline task — a ranking score, a summary, or a category label — on a fixed dataset, and none measures what happens once that model has to run, repeatedly, inside a live request handler that also combines its output with other signals (keyword and semantic matching, in Skyler AI's case) into one product decision. Prexawanprasut even deploys a prototype "real-time" system on top of its classifier, but reports no latency, serving-cost, or data-locality numbers for it — the same implicit production claim, left unmeasured, that motivated this paper's investigation.

One honest caveat on the "classification quality, measured by us" row: it is **not** a same-task comparison, for any of the three. Mukherjee & Jiang predict a ranking score for triage action; Zhang et al. summarize; Prexawanprasut classifies into business-workflow categories. Skyler AI's sentiment model is not trained or used for any of these three tasks — it classifies general sentiment (positive/negative) as one input to a user-configurable inbox rule. We report 89.91%/90.27% because it is the real, measured, standard-benchmark number for the *exact model Skyler AI actually runs*, not because it is directly comparable task-for-task to any of the three papers' numbers. We would rather show a real number on a different task, clearly labeled as such, than leave the cell blank or imply a false equivalence.

2.2 Small/local models for email classification (accuracy–cost tradeoff), broader literature

- **Rojas-Galeano (2024)**, *"Zero-Shot Spam Email Classification Using Pre-trained Large Language Models"* (arXiv:2405.15936; published in *Applied Computer Sciences in Engineering*, WEA 2024, Springer CCIS vol. 2222), compares a smaller open model (Flan-T5) against GPT-4 for zero-shot spam classification on SpamAssassin, reporting Flan-T5 at ~90% F1 versus GPT-4 at ~95%, and explicitly flags the deployment cost of proprietary models as a concern. The paper's contribution is about *classification quality under a model-size tradeoff*; it does not discuss how either model is kept warm or re-used across requests in a serving application.

- **Kamberi (2026)**, *"Intelligent Email Routing with LLMs"* (M.Sc. thesis, South East European University, supervised by Prof. Dr. Mennan Selimi), pairs a `sentence-transformers/all-MiniLM-L6-v2` embedding model (the same embedding model Skyler AI uses) with keyword extraction for email

routing, and compares it against larger GPT/LLaMA-based setups on a cost-versus-accuracy basis. Again, the focus is model selection and routing accuracy, not request-time inference lifecycle.

- A model card for **Distil Labs'** `distil-email-classifier` (a distilled ~0.8B-parameter Qwen3-based model, reported at ~93% accuracy across 10 categories) is offered explicitly as a *local, privacy-preserving* alternative to cloud classification — validating the same design motivation behind Skyler AI's local DistilBERT/MiniLM pipeline — but it is a model artifact description, not a study of serving behavior.

Observation: this thread, including both IEEE papers above, treats the local model as a black box evaluated once, offline, on a fixed dataset. None of it asks what happens when that model has to be initialized, held, and reused across many short-lived HTTP requests in a live backend, which is exactly the setting Skyler AI runs in.

2.3 Cold-start latency for LLM serving (systems–infrastructure)

- **Kabakibo, Trivedi, and Wang (2026)**, *"Breaking the Ice: Analyzing Cold Start Latency in vLLM"* (arXiv:2606.07362), profile the startup path of vLLM, a large-scale GPU LLM-serving engine, and find that startup cost is dominated by CPU-bound work rather than GPU or storage I/O, building an analytical model to predict startup time per hardware configuration.

- **Liu et al. (2025)**, *"CSGO: Generalized Optimization for Cold Start in Wireless Collaborative Edge LLM Systems"* (arXiv:2508.11287), address cold start in pipelined LLM inference distributed across wireless edge devices, overlapping model loading with computation/communication and formulating layer partitioning as a mixed-integer nonlinear program.

Observation: this thread is about *infrastructure-scale* serving — a single long-lived engine (vLLM) or a distributed edge pipeline — where "cold start" means the first request after a deploy or device join. It does not cover the much smaller, more common case of a lightweight in-process model (tens of megabytes, CPU/WASM inference, no dedicated serving engine) that an application re-initializes **every time a request handler runs**, because the application code — not an inference server — owns the model's lifecycle.

2.4 The gap this paper addresses

> Neither the direct literature baseline (Section 2.1), the broader accuracy/cost literature (Section 2.2), nor the infrastructure-scale cold-start literature (Section 2.3) evaluates the *compounding, per-request* cost of re-initializing a small local transformer pipeline *inside application request-handling code*, or compares sequential per-item versus batched inference for the hybrid semantic + keyword email-filtering pattern used by tools like Skyler AI, Kamberi's routing system, and Distil Labs' classifier. This is a serving-lifecycle gap that sits between the literatures above: too small-scale and too embedded in ordinary backend code to be "LLM serving" in the vLLM/edge sense, but outside the scope of the triage/summarization/classification papers, which assume the model is already loaded and measure only offline, single-task metrics.

This gap matters practically: applications built the way the direct baseline literature recommends (small local model, run per-message, combined into a product decision) are exactly the ones at risk of this bug, because the papers that validate the *model choice* — including Prexawanprasut's deployed prototype — give no guidance on the *model lifecycle*, or on whether the combined, multi-signal decision built on top of that model is itself correct.

---

3. Problem Statement

Concretely, in the pre-fix version of Skyler AI:

1. `apps/api/src/AI_Services/models.ts` defines a module-level singleton loader, `loadModels()`, guarded by `if (sentimentAnalyzer && embedder) return;` — the intended mechanism for loading

the DistilBERT and MiniLM pipelines exactly once per process.

2. `loadModels()` was **never called anywhere in the codebase.** It was dead code.

3. Instead, `apps/api/src/routes/gmail/webhooks/gmailPull.ts` — the handler behind `GET /api/gmail/pull/:instanceId`, which backs the product's "pull up to 200 mails" feature — did this on *every single request* that had an AI filter rule attached:

```ts
const classifier = new EmailClassifier();
await classifier.init();
```

and `EmailClassifier.init()` called `pipeline(...)` for both models directly, from scratch, discarding the result at the end of the request.

4. Separately, `EmailClassifier.filterEmails()`'s semantic-matching path (`semanticAnyMatch`) split each email into sentences and called the embedder once per sentence, in a sequential `for` loop with `await` inside it, rather than batching all of an email's sentences (or all filter keywords) into a single call.

Both issues are invisible in isolated unit testing (there was no test exercising this path at all — see Section 6.2) and only show up as latency under realistic load, which matches the "too slow, too buggy" feedback that motivated this investigation.

---

4. System Overview

Skyler AI ("Skyler Inbox") is a Gmail triage and reply-assist tool built as a pnpm/Turborepo monorepo:

- **Frontend:** Next.js, NextAuth (Google OAuth), Tailwind, shadcn/ui, Dexie (client-side cache of pulled emails).

- **Backend:** Express + TypeScript (`apps/api`).

- **Database:** Supabase/Postgres via Prisma ORM.

- **Local NLP models** (`apps/api/src/AI_Services`): `Xenova/distilbert-base-uncased-finetuned-sst-2-english` for sentiment, `Xenova/all-MiniLM-L6-v2` for sentence embeddings, both served through `@xenova/transformers` (ONNX runtime, CPU/WASM, no GPU required).

- **Cloud LLM:** Groq `llama-3.1-8b-instant` via LangChain (`GmailSummariser`), used only for the smaller, user-initiated task of summarizing one email and drafting a reply against a user-defined rule set.

- **Payments / credits:** a credit-based freemium system (Cashfree, PayPal) gates how many "pulls" a user can perform per month.

The relevant request path is:

```
GET /api/gmail/pull/:instanceId

→ fetch up to N Gmail messages (batched, 10 at a time, via googleapis)

→ if the inbox instance has an AI filter rule:

new EmailClassifier() → init() → filterEmails(messages, rules)

→ return filtered messages to the client
```

```

Because every inbox "refresh" costs the user one credit regardless of how many emails are classified, the server-side cost of classification directly affects both perceived responsiveness and infrastructure cost per credit.

---

## 5. Proposed Improvement

Grounded in the gap identified in Section 2.4, we implement three changes, all scoped to the AI classification path and requiring no breaking change to the product's existing behavior or API contract:

### 5.1 Warm, process-wide model singleton

`EmailClassifier.init()` now calls the existing (previously unused) `loadModels()` singleton loader from `models.ts` and reads the shared `sentimentAnalyzer` / `embedder` references, instead of calling `pipeline(...)` itself:

```ts

async init() {

await loadModels();

this.sentimentAnalyzer = Models.sentimentAnalyzer;

this.embedder = Models.embedder;

}

```

`loadModels()`'s existing guard (`if (sentimentAnalyzer && embedder) return;`) means every call after the first resolves immediately. We additionally call `loadModels()` once at process startup (`server.ts`), so the *first* real request is also warm, rather than paying the load cost on whichever user happens to trigger it first.

5.2 Batched embedding inference (kept, with a measured caveat)

We replace the per-sentence embedding loop with a single batched call per email (and a single batched call for the filter rule's keyword list), using `@xenova/transformers`' native support for array input to the `feature-extraction` pipeline. A new `asBatch3D` helper unpacks the resulting `[batch, seq, hidden]` tensor into one token matrix per input text, which is then mean/CLS-pooled exactly as before — the pooling and cosine-similarity logic, and therefore the filtering semantics, are unchanged. We verified this with a focused unit test suite (Section 6.2) rather than relying on inspection alone.

We kept this change in production, but Section 6.3/7 report an important caveat we only found by testing at realistic scale: batching this call gives a clean speedup only when the batched texts are of similar length (e.g., a short filter-rule keyword list); on the variable-length sentences real email text actually produces, the benefit mostly disappears due to padding overhead. We left it in rather than reverting it because, unlike the sentiment case below, it was never measured to be a net *regression* — at worst a wash.

5.2b An optimization we tried and reverted: batched sentiment analysis

Following the same reasoning as 5.2, we also tried batching the sentiment-analysis call — one call for every email in a request instead of one call per email — on the theory that it should help the same way. We shipped it, wrote a test for it, and then, while building the realistic-scale benchmark in Section 6.3, measured it directly against the original per-email pattern at 200 variable-length synthetic emails, five repeated trials: batched sentiment analysis was **consistently ~8% slower**, in all five trials, than one call per email. The likely cause is the same padding mechanism as 5.2, just large enough here to turn a null result into a
```

measured regression. We reverted this change — `EmailClassifier.filterEmails()` calls `analyzeSentiment()` once per email again — rather than ship an "optimization" that our own measurement showed makes the product slower. The reverted attempt, its measurement, and the revert are all preserved in `apps/api/src/scripts/benchmarkClassifier.ts`'s commit history and in the code comment left at the call site, so the decision is auditable rather than silently undone.

5.3 In-app production latency telemetry

Both 5.1 and 5.2 were measured offline (Section 6–7). To close the "no end-to-end production measurement" gap honestly rather than just noting it as a limitation, we also instrument the real request path: `apps/api/src/routes/gmail/webhooks/gmailPull.ts` now times the `classifier.init()` + `filterEmails()` call per request, logs `classified N emails -> M kept in <ms>ms` server-side, and returns the same `classifyMs` figure to the client in the JSON response. This is a small, additive change (one new numeric field; no existing field removed or renamed) that turns "the fix should make requests faster" into something anyone running the deployed app — not just someone re-running our benchmark script — can observe directly, e.g. in server logs or by inspecting the network response in the browser.

All of these are internal refactors/additions, including the one we reverted: the public *filtering* behavior of `filterEmails()` (which emails are kept, under which rules) is unchanged throughout; only *how* the models are obtained and called changes, plus one new, additive response field.

---

6. Implementation

The changes touch five files:

- `apps/api/src/AI_Services/models.ts` — unchanged in content; now actually used.

- `apps/api/src/AI_Services/gmail_AI/emailClassifier.ts` — `init()` now delegates to the shared singleton; `getVector` / `as2D` (single-item embedding path) were removed as dead code once every call site moved to the new batched `getVectors` / `asBatch3D` path; sentiment analysis stayed per-email after the batching attempt in Section 5.2b was reverted.

- `apps/api/src/server.ts` — calls `loadModels()` once at boot, logging success/failure without blocking the HTTP listener from starting.

- `apps/api/src/routes/gmail/webhooks/gmailPull.ts` — times the classification call and returns `classifyMs` (Section 5.3).

- `apps/api/src/scripts/benchmarkClassifier.ts` — the checked-in, reproducible benchmark harness described below (new).

6.1 Isolated, mechanism-level benchmarks

To isolate each mechanism from the others and from end-to-end noise, we first measured each change on its own, directly against `@xenova/transformers`' `pipeline()` function, with no mocks:

1. **Re-initialization cost.** Call `pipeline(...)` for both models twice in sequence (simulating two consecutive `/gmail/pull` requests under the old code), then compare against reusing already-resolved references (the new code's behavior after the first warm-up). Run on the project's own cached ONNX weights (`node_modules/.pnpm/@xenova+transformers@2.17.2/.../\.cache`), i.e. with **no network/model-download cost** — a conservative choice, since a real cold container would pay a download cost on top of this.

2. **Embedding batching, uniform-length text.** Embed 30 near-identical-length synthetic sentences one call at a time versus one batched call.

3. **Embedding batching, realistic variable-length text.** The same comparison, but on ~600 sentences built from 10 email-like templates of genuinely different lengths (matching the sentence count of ~200 realistic emails).

4. **Sentiment-analysis batching, realistic variable-length text.** The same sequential-vs-batched comparison applied to the sentiment pipeline, over 200 email-like texts, 5 repeated trials.

Experiments 1–2 were single-shot measurements on a warm model; experiments 3–4 (added after the result of 2 turned out not to generalize) were run for 5 repeated trials each to check consistency before deciding whether to ship or revert.

6.2 Correctness verification

Because this is a refactor of logic that was previously untested, we added `apps/api/src/__tests__/emailClassifier.test.ts`, which mocks the model layer with deterministic fake embeddings/sentiment labels and checks that: (a) sentiment-based filtering keeps the right emails, (b) literal keyword matching works with semantic analysis off, (c) semantic matching finds a conceptually related email that does not contain the literal keyword, (d) the embedder is called a constant number of times per request (one batch per email's sentences, one batch for the keyword list) rather than once per sentence/keyword, and (e) the sentiment analyzer is called once *per email* (pinning down the reverted, not the batched, behavior, so it can't silently regress back to the measured-worse pattern). We also ran an equivalent check directly against the real (non-mocked) class with injected fake model functions, bypassing the project's test harness (which needs a live Supabase database unavailable in this environment), to confirm the same behavior outside Jest.

6.3 Realistic-scale, end-to-end benchmark

Sections 6.1's isolated numbers don't tell us what a real `/gmail/pull` request feels like, so we also built `apps/api/src/scripts/benchmarkClassifier.ts`, a reproducible, checked-in harness that:

- Generates 200 synthetic-but-realistic emails (the product's own stated "pull up to 200 mails" limit), built from 10 varied templates (refunds, meetings, invoices, deadlines, receipts, etc.) so sentence lengths are genuinely non-uniform, unlike the toy test in 6.1.2.

- Runs the **actual, unmodified** `EmailClassifier` from the codebase against `OldEmailClassifier`, a verbatim reconstruction of the pre-fix class recovered from git history (commit `50753ab^`), kept only in this benchmark file — never reintroduced into production code.

- Times each simulated request exactly the way the real endpoint does: `t0 → new classifier → init() → filterEmails() → t1`, matching `classifyMs` in `gmailPull.ts` precisely.

- Repeats this for multiple requests per trial and multiple trials, reporting mean, median, standard deviation, and max — not just a mean from one run — because, as Section 7 shows, the mean alone was misleading on this machine.

Run it yourself with `cd apps/api && npx ts-node --compiler-options '{"module":"commonjs"}' src/scripts/benchmarkClassifier.ts` (takes a few minutes; `BENCH_EMAILS`, `BENCH_REQUESTS`, `BENCH_TRIALS` env vars adjust scale).

---

7. Results

7.1 Isolated, mechanism-level results (Section 6.1)

Measurement	Old behavior	New behavior	Notes
Model (re)initialization cost paid on a classification	252–310 ms **every request**	~0 ms after the first request (warmed at	Local disk-cached ONNX weights; a cold network fetch would make the "old" number

Measurement	Old behavior	New behavior	Notes
request		boot)	larger. The one clean, unambiguous, repeatable win.
Embedding 30 **uniform-length** synthetic sentences	56.9 ms (sequential)	36.7 ms (batched)	**1.55x** speedup — but this result did not generalize (see next row)
Embedding ~600 **variable-length** realistic sentences (5 trials)	mean 1027.3 ms (sequential)	mean 1056.9 ms (batched)	**0.97x** — a wash, slightly negative. Padding to the batch's longest sentence erodes the batching benefit once lengths vary.
Sentiment analysis on 200 **variable-length** realistic emails (5 trials)	mean 1054.7 ms (sequential, all 5 trials: 1039–1095 ms)	mean 1142.4 ms (batched, all 5 trials: 1125–1159 ms)	**0.92x — consistently worse in all 5 trials, no overlap between the two distributions.** Reverted from production (Section 5.2b).

The sentiment-batching result is the most important row in this table precisely because it is negative: it is evidence that we measured rather than assumed our optimizations, and it directly explains why the realistic end-to-end numbers below (7.2) are smaller and noisier than a naive extrapolation from the first row would suggest.

This is also exactly the kind of number the direct baseline literature in Section 2.1 does not report: none of the three papers studied measures what their chosen model costs to load or serve on a live, request-driven backend — each reports only an offline ranking score, a summary-quality discussion, or a classification accuracy, computed once on a fixed dataset. Our 252–310 ms-per-request figure is the missing serving-side counterpart to that kind of offline result.

7.2 Realistic-scale, end-to-end results (Section 6.3): three independent runs

We ran the full `benchmarkClassifier.ts` harness (200 emails/request, 3 requests/trial, 5 trials/run — 15 samples per classifier per run) three times, independently, on the same machine, on different occasions. We report all three runs rather than the most favorable one:

Run	OLD mean (median / stdev / max), ms	NEW mean (median / stdev / max), ms	Speedup (mean / median)	Variance reduction (stdev)
1	5970 (4502 / 2733 / 12820)	4367 (4295 / 333 / 5494)	1.37x / 1.05x	**8.2x**
2	5003 (4262 / 2456 / 14020)	4541 (4210 / 899 / 7659)	1.10x / 1.01x	2.73x
3	5176 (4779 / 1190 / 8034)	5112 (4754 / 974 / 8059)	1.01x / 1.01x	1.22x

Two things are true at once, and we report both rather than picking the one that makes a better headline:

- **The mean/median speedup at this macro scale is inconsistent across runs** (1.37x down to essentially 1.00x), and on this particular shared/sandboxed machine, background system noise at the scale of several seconds per request is large enough to sometimes swamp a ~250–310 ms fixed saving. We do not trust any single one of these three numbers as "the" speedup, which is exactly why we ran it three times and report all three (see Limitations, Section 8).

- **The reduction in variance (standard deviation) is large and consistent in direction across all three runs** (1.2x–8.2x), and the reduction in worst-case latency is large in every run we observed (run 1's OLD worst case, 12.8 s, is roughly `2.3x` run 1's NEW worst case, 5.5 s). The old, buggy code occasionally produced severe latency spikes (11.8 s, 12.8 s, 14.0 s — roughly 3x a typical request) that we attribute to variable-cost ONNX session (re-)creation under the old per-request re-init pattern; the new code's worst observed case across all three runs (8059 ms) is still bounded well below the old code's worst cases. **The fix's clearest, most defensible real-world benefit at this scale is making request latency more**

predictable and eliminating severe tail-latency spikes, not a fixed multiplier on the typical-case request.

7.3 Outcome: in-app telemetry on live traffic

As described in Section 5.3, `GET /api/gmail/pull/:instanceId` now returns a `classifyMs` field and logs the same figure server-side whenever AI filtering runs. This is a concrete, shipped outcome beyond the offline benchmark: once Skyler AI is run against a real account (outside this sandboxed environment), `classifyMs` should be observable to drop from "roughly 250–310 ms plus embedding time" to "roughly just embedding time" between the pre-fix and post-fix code, without re-running any special benchmarking script — closing part of the "no end-to-end production measurement" limitation from this evaluation (Section 8) with a feature now in the codebase rather than only a benchmark script outside it.

7.4 Classification quality, measured against a real benchmark (not left blank)

A latency fix alone invites an obvious question: does Skyler AI's underlying model actually classify anything well? Rather than cite a number from somewhere else, or leave it blank because it wasn't the paper's original focus, we measured it directly.

Method. We fetched the full, official **SST-2 validation split** (872 labeled examples: `stanfordnlp/sst2` on Hugging Face, accessed via the public `datasets-server.huggingface.co` API) and ran it through `Xenova/distilbert-base-uncased-finetuned-sst-2-english` — not a re-implementation or a similar model, the exact ONNX model file `apps/api/src/AI_Services/models.ts` loads in production — via `apps/api/src/scripts/evalSentimentModel.ts` (new, checked in). SST-2 is the standard benchmark this model family is fine-tuned and reported on, which is why we chose it over an ad hoc dataset of our own.

Result:

Metric	Value
Accuracy	**89.91%** (784/872 correct)
Precision	88.70%
Recall	91.89%
F1	90.27%
Confusion matrix	TP=408, FP=52, TN=376, FN=36

This is close to, but a little below, the ~91–92% accuracy commonly reported for the original (non-quantized, PyTorch) `distilbert-base-uncased-finetuned-sst-2-english` checkpoint — consistent with a small, expected accuracy cost from the ONNX quantization `@xenova/transformers` uses to make this model cheap enough to run per-request in a Node.js backend with no GPU. We see that cost as evidence the number is real, not cherry-picked: we did not expect or look for 89.91% specifically, we ran the one available standard benchmark and reported what came out, quantization cost included.

What this number is not: it is not a triage-action AUROC, a summarization score, or a workflow-classification accuracy, and it is not directly comparable, task-for-task, to any of the three papers' numbers in Section 2 — see the caveat after the comparison table in Section 2.1. It is the real, measured quality of the one component of Skyler AI's pipeline (sentiment classification) that has a standard public benchmark to measure against. The semantic/keyword-matching component (MiniLM) and the overall email-filtering outcome (which emails a user's rule keeps) do not have an equivalent standard, labeled, public benchmark we could run against — measuring those honestly would need a labeled corpus of real emails with ground-truth "should this rule keep this email?" labels, which does not exist publicly (see Section 8 and, for a self-constructed version of exactly such a labeled set, Section 11's Addendum).

---

8. Discussion and Limitations

- **Benchmark rigor (partially addressed, and partially the point).** Section 7.1's isolated results use 5 repeated trials with tight, non-overlapping distributions (sentiment batching: 1039–1095 ms sequential vs. 1125–1159 ms batched, every time) and are trustworthy. Section 7.2's realistic end-to-end results use 3 independent 5-trial runs (15 samples/run) rather than one run, specifically *because* a single run would have overstated the effect (run 1 alone would have claimed a clean 1.37x with 8.2x less variance; run 3 alone would have claimed almost nothing). All numbers are still from one developer machine, not a statistically controlled benchmark across hardware, and we did not compute formal confidence intervals. A stronger version of this paper would run each configuration N≥30 times, report confidence intervals, and test on at least one additional machine class (e.g., a small cloud instance similar to where Skyler AI is actually deployed, per `render.yaml`) to separate "noisy sandboxed laptop" variance from a true property of the fix.

- **Scope of "cold start."** We did not measure the very first model load after a fresh `pnpm install` (i.e., including a real network download of the ONNX weights), which would make the "old" numbers in Section 7 even larger and is arguably the more realistic worst case for a newly deployed container.

- **No end-to-end production measurement (partially addressed).** We benchmarked the classification stage in isolation; we did not measure total `/api/gmail/pull/:instanceId` latency against live Gmail and Supabase, which also includes network calls to the Gmail API that this change does not touch. The `classifyMs` telemetry added in Section 5.3/7.3 makes this measurable going forward without further code changes, but we have not yet collected that live data ourselves — this environment has no access to the production database or a real Gmail OAuth session.

- **Generality.** The specific gap we describe (Section 2.4) is argued from a targeted literature search, not a systematic review; it is plausible that a paper covering exactly this "in-process small-model request lifecycle" problem exists under different keywords (e.g., "model caching", "warm pool", "singleton inference service") in the broader MLOps/model-serving literature outside the email domain, and a full related-work section for publication should search that space explicitly.

- **Fairness of comparison.** Because `@xenova/transformers` pipelines are not literally re-downloaded on a warm machine (only re-initialized from already-cached weights into memory/ONNX session objects), the 252–310 ms figure already understates the cost of the bug in a cold-container deployment, but it is the correct, honest number for *this* benchmark's conditions, and we report it as such rather than extrapolating an unmeasured worst case.

- **No task-matched classification benchmark, at first.** Section 7.4's 89.91%/90.27% is real and measured, but on general sentiment (SST-2), not the triage, summarization, or workflow-classification tasks evaluated in Section 2 — at the time that number was first measured, there was no public, labeled "should Skyler AI's rule keep this real email" dataset to measure the actual product decision against, so that number was a sentiment-model quality check, not an end-to-end filtering-accuracy claim. Section 11's Addendum closes this gap with a self-constructed labeled set and a measured end-to-end result, though that set is still not a public, independently-curated benchmark — see the Addendum's own caveat.

---

9. Outcomes Summary

For a reader looking for a single answer to "what did this project actually produce":

1. **A verified gap, not an assumed one.** Three real, freely-downloadable papers (Mukherjee & Jiang on triage-action prediction; Zhang et al.'s EmailSum on thread summarization; Prexawanprasut on business-workflow classification) were read directly from their public sources — arXiv, the ACL anthology, and Thailand's open-access TCI-Thaijo platform, no paywall or institutional login needed for any of them. None measures request-time serving behavior or a combined, multi-signal decision, despite one (Prexawanprasut) explicitly deploying a "real-time" prototype workflow system on top of its classifier.

2. **A real bug, fixed in shipped code**, not a synthetic example: `EmailClassifier` re-initialized two ONNX models on every filtered `/gmail/pull` request; this is now a warm, process-wide singleton (Section 5.1). This is the paper's one clean, unambiguous, repeatable result: 252–310 ms removed from every classification request, confirmed in isolation with tight, non-overlapping measurements.

3. **A new, additive production feature**: `classifyMs` telemetry on the live endpoint (Section 5.3), so the claimed improvement is checkable on real traffic, not only in a benchmark script.

4. **A reproducible, checked-in benchmark harness** (`apps/api/src/scripts/benchmarkClassifier.ts`) that drives the actual production classifier at the product's own realistic scale (200 emails/request), not a toy example — and that is what surfaced the two findings below.

5. **An honest negative result, acted on rather than hidden**: batching the sentiment-analysis call looked like it should help (it mirrors the embedding fix) but measured as a consistent ~8% *regression* at realistic scale, in all 5 repeated trials with no overlap between the "faster" and "slower" distributions. We reverted it. The embedding-batching fix was kept, but only because it was never measured as net-negative, not because it was proven to reliably help — at realistic variable-length text, it is a wash (0.97x), not the 1.55x its uniform-length toy test showed.

6. **A transparent, multi-run macro benchmark** rather than one favorable number: three independent 200-email, 5-trial runs gave mean speedups of 1.37x, 1.10x, and 1.01x — noisy and inconsistent on this machine — but a consistent 1.2x–8.2x reduction in latency *variance*, and a visibly bounded worst case (old code's tail spikes reached 11.8–14.0 s; new code's worst observed case across all three runs was 8.1 s). The defensible real-world claim is "more predictable, fewer severe spikes," not a fixed multiplier.

7. **Verified correctness**, not just verified speed: a unit test suite (including a test that pins down the *reverted* sentiment behavior so it can't silently regress) plus a standalone cross-check confirm the filtering *results* are unchanged throughout (Section 6.2).

8. **A real classification-quality number where the comparison table would otherwise read "—"**: 89.91% accuracy / 90.27% F1, measured by actually running Skyler AI's deployed sentiment model against the real, official 872-example SST-2 validation set (Section 7.4) — not copied from a model card, not estimated, and not on the same task as any of the three baseline papers' numbers, which we say plainly rather than imply a false equivalence.

10. Conclusion and Future Work

We identified a concrete, shipped instance of a serving-lifecycle gap in small-model NLP applications — re-initializing a local transformer pipeline on every API request, instead of once per process — that sits outside a direct, verified literature baseline covering triage, summarization, and workflow classification (Section 2.1), the broader accuracy/cost-focused literature on local email classification models (Section 2.2), and the infrastructure-scale cold-start literature for large LLM serving engines (Section 2.3). We fixed this in Skyler AI by introducing a warm model singleton, added in-app telemetry so the effect is observable on real traffic, and built a reproducible benchmark harness driving the real production classifier at realistic scale. That harness also did its job as a check on our own work: it showed that a second optimization (batched sentence embedding) only helps on unrealistically uniform-length text, and that a third (batched sentiment analysis) is a measured regression, which we reverted rather than ship. The one number we are confident in without qualification is the 252–310 ms removed per request by eliminating re-initialization; the realistic, end-to-end effect of that fix is real but noisier to measure precisely on a shared machine, and shows up most clearly as reduced tail-latency variance rather than a clean multiplier.

Future work: (1) a statistically rigorous, multi-machine benchmark on dedicated (non-shared) hardware, including a true cold-network-fetch scenario, to get a trustworthy macro-scale multiplier instead of the noisy 1.01x–1.37x range reported here; (2) collecting real `classifyMs` data from live Gmail/Supabase traffic now

that the telemetry exists; (3) investigating length-bucketed batching (sorting sentences/emails by length before batching, to reduce the padding overhead that erased the embedding-batching win and caused the sentiment-batching regression) as a follow-up that might recover a real benefit from batching without the measured downside; (4) generalizing the singleton-reuse fix into a small reusable "warm model pool" library, since the same pattern is likely to recur as more productivity tools adopt local/hybrid NLP pipelines for privacy reasons, per the motivation in Rojas-Galeano (2024), Kamberi (2026), the Distil Labs model card, and the direct baseline papers in Section 2.1; (5) a held-out, real-email-traffic validation of the filtering-accuracy benchmark introduced in Section 11, now that all three direct baseline papers are fully open-access and no further literature access is blocking that work.

References

1. Mukherjee, S., & Jiang, K. (2019). *A Content-Based Approach to Email Triage Action Prediction: Exploration and Evaluation.* arXiv:1905.01991.

2. Zhang, S., Celikyilmaz, A., Gao, J., & Bansal, M. (2021). *EmailSum: Abstractive Email Thread Summarization.* Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP 2021). arXiv:2107.14691.

3. Rojas-Galeano, S. (2024). *Zero-Shot Spam Email Classification Using Pre-trained Large Language Models.* In: Applied Computer Sciences in Engineering (WEA 2024), Communications in Computer and Information Science, vol. 2222, Springer. arXiv:2405.15936.

4. Kamberi, H. (2026). *Intelligent Email Routing with LLMs.* M.Sc. Thesis, South East European University, Faculty of Contemporary Sciences & Technologies. Supervised by Prof. Dr. Mennan Selimi.

5. Kabakibo, H. S., Trivedi, A., & Wang, L. (2026). *Breaking the Ice: Analyzing Cold Start Latency in vLLM.* arXiv:2606.07362.

6. Liu, X., Xue, N., Bao, R., Sun, Y., Chen, Z., Tao, M., Xu, X., & Cui, S. (2025). *CSGO: Generalized Optimization for Cold Start in Wireless Collaborative Edge LLM Systems.* arXiv:2508.11287.

7. Distil Labs. *distil-email-classifier* (model card). A distilled Qwen3-based local email classifier, reported ~93% accuracy across 10 categories. Accessed via featherless.ai.

8. Xenova / Hugging Face. `distilbert-base-uncased-finetuned-sst-2-english` and `all-MiniLM-L6-v2` (ONNX ports used by `@xenova/transformers`), used as the local models under study in this paper.

9. Prexawanprasut, T. (2025). *Transformer-Based Email Classification for Workflow Automation in Small and Medium-Sized Enterprises.* Thailand Statistician. Open access via TCI-Thaijo. Reports XLM-R exceeding 92% accuracy on 12,500 business emails across 4 workflow categories, integrated into a prototype real-time system, with no latency, serving-cost, or privacy measurement given anywhere in the paper — the same pattern identified in references 1–2 (Section 2.1/4, Addendum).

> **On source access:** references 1, 2, and 9 — the direct baseline — are all freely downloadable with no institutional login or paywall: references 1 and 2 from arXiv/the ACL anthology, reference 9 from Thailand's open-access TCI-Thaijo journal platform. Their quoted metrics (AUROC, dataset size, accuracy) were read directly from the papers' own text, not a search-engine summary. References 3–6 were likewise confirmed against arXiv/thesis-repository pages rather than quoted from search-engine summaries alone.

Appendix A — Reproducing the benchmarks

Unlike an earlier draft of this paper, the realistic-scale benchmark is a real, checked-in, version-controlled file, not a throwaway snippet reproduced only in this document:

[`apps/api/src/scripts/benchmarkClassifier.ts`](../apps/api/src/scripts/benchmarkClassifier.ts). It contains, in full, with comments:

- The exact pre-fix `OldEmailClassifier`, recovered verbatim from git history (`git show 50753ab^:apps/api/src/AI_Services/gmail_AI/emailClassifier.ts`), kept only in this file for comparison.

- The synthetic-but-realistic 200-email generator used for Section 6.3/7.2.

- The timing harness (mean/median/stdev/max) used to produce the Section 7.2 table.

Reproduce it with:

```bash

cd apps/api

npx ts-node --compiler-options '{"module":"commonjs"}' src/scripts/benchmarkClassifier.ts

# optional: BENCH_EMAILS=200 BENCH_REQUESTS=3 BENCH_TRIALS=5 npx ts-node ... (same command)

```

The isolated micro-benchmarks in Section 6.1/7.1 (re-initialization cost; uniform- vs. variable-length embedding batching; sentiment batching) were smaller, throwaway scripts written directly against `@xenova/transformers` ' `pipeline()` function with no mocks, run outside the repository; their logic is described in Section 6.1 precisely enough to reconstruct, and is intentionally not duplicated as a second checked-in script to avoid maintaining two near-identical harnesses — `benchmarkClassifier.ts` is the one meant to be re-run and extended.

Appendix B — Files changed

- `apps/api/src/AI_Services/gmail_AI/emailClassifier.ts` — singleton reuse; length-bucketed batching for both embedding and sentiment (see Addendum, Section 11); a measured fix to the default semantic-similarity threshold (0.6 → 0.35); a new `filterEmailsWithReasons()` method for explainability and accuracy evaluation, alongside the unchanged `filterEmails()` contract

- `apps/api/src/AI_Services/models.ts` (no content change; now invoked)

- `apps/api/src/server.ts` — warms models at boot

- `apps/api/src/routes/gmail/webhooks/gmailPull.ts` — `classifyMs` telemetry (Section 5.3/7.3); live endpoint now returns `filterEmailsWithReasons()` 's additive `reason` field per email (Addendum, Section 11)

- `apps/api/src/scripts/benchmarkClassifier.ts` (new) — reproducible realistic-scale latency benchmark, Section 6.3/7.2

- `apps/api/src/scripts/evalSentimentModel.ts` (new) — classification-quality benchmark against the real SST-2 validation set, Section 7.4

- `apps/api/src/scripts/evalFilteringAccuracy.ts` (new) — end-to-end filtering-decision accuracy benchmark against a labeled test set; see Addendum, Section 11

- `apps/api/src/scripts/verifyNoNetworkEgress.ts` (new) — proves zero network calls during model load and classification; see Addendum, Section 11

- `apps/api/src/__tests__/emailClassifier.test.ts` (new) — correctness tests for batching, sentiment, and the explainability method

11. Addendum — Findings After Initial Submission

Two further rounds of measurement, after the results above were first written up, changed what's actually shipped. Reported here rather than silently folded into the sections above, so the paper's own history stays auditable.

Length-bucketed batching (resolves the Section 5.2b revert). The naive full-batch sentiment call described in 5.2b was a measured regression and was reverted to per-email calls. A follow-up experiment tried *length-bucketed* batching instead — sort texts by length, batch within groups of ≤16 — on the theory that naive batching's problem was padding waste from mixing short and long texts in one call, not batching itself. Measured in 3-trial isolated tests with clean, non-overlapping distributions: ~1.14x for sentiment, ~1.21–1.24x for embedding, beating both sequential calls and the naive full batch. This is now shipped (`callInLengthBuckets()` in `emailClassifier.ts`), replacing both the reverted naive-sentiment-batch attempt and the original per-email sentiment calls.

A real filtering-accuracy bug, found and fixed. Neither IEEE baseline, nor any cited work, nor the original Skyler AI code, had ever measured whether the product's actual multi-signal decision (sentiment AND (keyword OR semantic similarity)) correctly keeps/discards a real email — only single-model, single-label accuracy had been measured anywhere in this comparison. A new labeled test set (90 synthetic emails, ground truth assigned by construction across a sentiment × topic grid, not human eyeballing — see `apps/api/src/scripts/evalFilteringAccuracy.ts`) was run against the real, deployed `EmailClassifier`. Result: the shipped default `similarityThreshold` of 0.6 gave 100% precision but only 43.3% recall (F1 60.5%) — verified by direct cosine-similarity measurement that genuinely on-topic paraphrases (no literal keyword) score 0.25–0.54 against single-word keywords, mostly below the 0.6 cutoff, so the semantic-matching path almost never fired in practice and the rule silently degraded to keyword-only matching. A threshold sweep (0.6 → 0.25) found 0.35 as the measured-best point: 94.4% accuracy, 100% precision (no regression), 83.3% recall, 90.9% F1 — a +30.4 F1-point fix, now shipped as the new default.

A new `filterEmailsWithReasons()` method was added alongside the unchanged `filterEmails()` to support this measurement. It is now also wired directly into the live `/gmail/pull` endpoint: every email in the response carries an additive `reason` field (sentiment label, matched keyword, semantic-match boolean) for the actual combined decision, not just one label — explainability none of the three direct-baseline papers (references 1, 2, 9) provide for a multi-signal rule. The existing response fields are unchanged; `reason` is additive only.

A proven, not claimed, privacy guarantee. None of the three papers in Section 2 measures or proves anything about data locality, despite privacy being a stated reason (in this project's own README and Section 4) to prefer small local models over cloud LLMs. `apps/api/src/scripts/verifyNoNetworkEgress.ts` patches every Node.js network entry point (`http` / `https` request and get, `net.connect`, `tls.connect`, global `fetch`) *before* loading the classification models, runs a full model-load-plus-classification cycle on realistic email text, and asserts zero network calls occurred anywhere in that cycle. Run: **PASS, 0 calls recorded** across model loading, sentiment analysis, and semantic embedding. This is also the empirical basis for ruling out TypeSafe AI's Jev (Section 2, "other candidate technologies" discussion in conversation with the project owner): any system requiring a network call per classification is categorically disqualified by this same test, which Skyler AI's current pipeline passes and a Jev-based pipeline could not.

A condensed, multi-page version of these findings, restructured as a direct comparison against the three cited papers (not an internal before/after), is published separately as `paper/outcomes-summary.pdf`.